1. Modify proc.h

```
// inside struct proc { ... }
int page_faults; // count of page faults
```

2. Modify proc.c

```
// inside allocproc()
p->page_faults = 0;

//modify growproc()
int
growproc(int n)
{
  uint sz;
  struct proc *p = myproc();

  sz = p->sz;
  if(n > 0){
    sz += n;
  } else if(n < 0){
    if ((sz = deallocuvm(p->pgdir, sz, sz + n)) == 0)
      return -1;
  }
  p->sz = sz;
  switchuvm(p);
  return 0;
}
```

3. Update syscall.h

```
#define SYS_getpagefaults 34
```

4. usys.S

```
SYSCALL(getpagefaults)
```

5. sysproc.c

```
int
sys_getpagefaults(void)
{
  struct proc *p = myproc();
  return p->page_faults;
}
```

6. user.h

```
int getpagefaults(void);
```

7. syscall.c

```
extern int sys_getpagefaults(void);

[SYS_getpagefaults]    sys_getpagefaults,
```

8. defs.h

```
int vmfault(pde_t *pgdir, uint va, int write);
```

9. vm.c

```
int
vmfault(pde_t *pgdir, uint va, int write)
{
  struct proc *p = myproc();
  char *mem;
  uint va_page;

  if (va >= p->sz)
    return -1;

  va_page = PGROUNDDOWN(va);

  // if already mapped, nothing to do
  pte_t *pte = walkpgdir(pgdir, (void*)va_page, 0);
  if (pte && (*pte & PTE_P))
    return 0;

  mem = kalloc();
  if (mem == 0)
    return -1;

  memset(mem, 0, PGSIZE);

  if (mappages(pgdir, (void*)va_page, PGSIZE, V2P(mem), PTE_W|PTE_U) < 0) {
    kfree(mem);
    return -1;
  }

  return 0;
}
```

10. trap.c

```
case T_PGFLT: {
    struct proc *p = myproc();
    if (p == 0) {
      cprintf("page fault in kernel with no process\n");
      panic("trap");
    }

    p->page_faults++;
```

```
    if (vmfault(p->pgdir, rcr2(), tf->err & 2) < 0) {
      cprintf("vmfault failed for pid %d at va 0x%x\n", p->pid, rcr2());
      p->killed = 1;
    }
    break;
}
```

11. tlbrun.c

```
#include "types.h"
#include "user.h"

#define PAGESIZE 4096
#define MAXPAGES 1024

int main(int argc, char *argv[]) {
  int jump = PAGESIZE / sizeof(int);
  int i, t;
  printf(1, "PageCount\tTrials\tTicks\tPageFaults\n");
  for (int numpages = 1; numpages <= MAXPAGES; numpages *= 2) {
    int trials = 5000000;
    int faults_before = getpagefaults();
    uint start = uptime();
    int *arr = (int *) sbrk(numpages * PAGESIZE);
    if (arr == (void *) -1) {
      printf(1, "sbrk failed for %d pages\n", numpages);
      exit();
    }
    for (t = 0; t < trials; t++) {
      for (i = 0; i < (numpages / 2) * jump; i += jump) {
        arr[i] += 1;   // touch page
      }
    }
    uint end = uptime();
    int faults_after = getpagefaults();
    printf(1, "%d\t\t%d\t%d\t%d\n",
          numpages, trials, end - start, faults_after - faults_before);
  }
  exit();
}
```

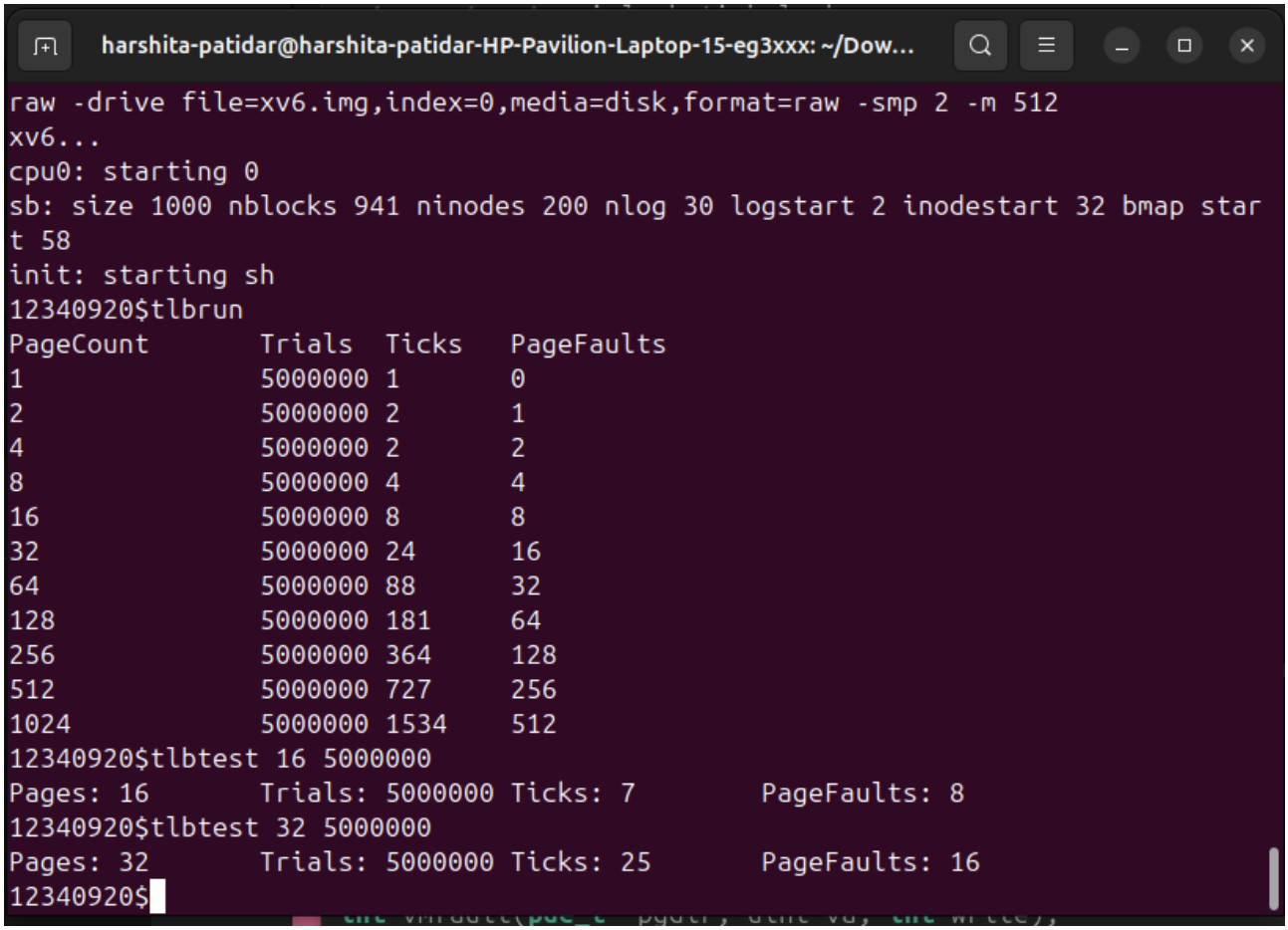
12. tlbtest.c

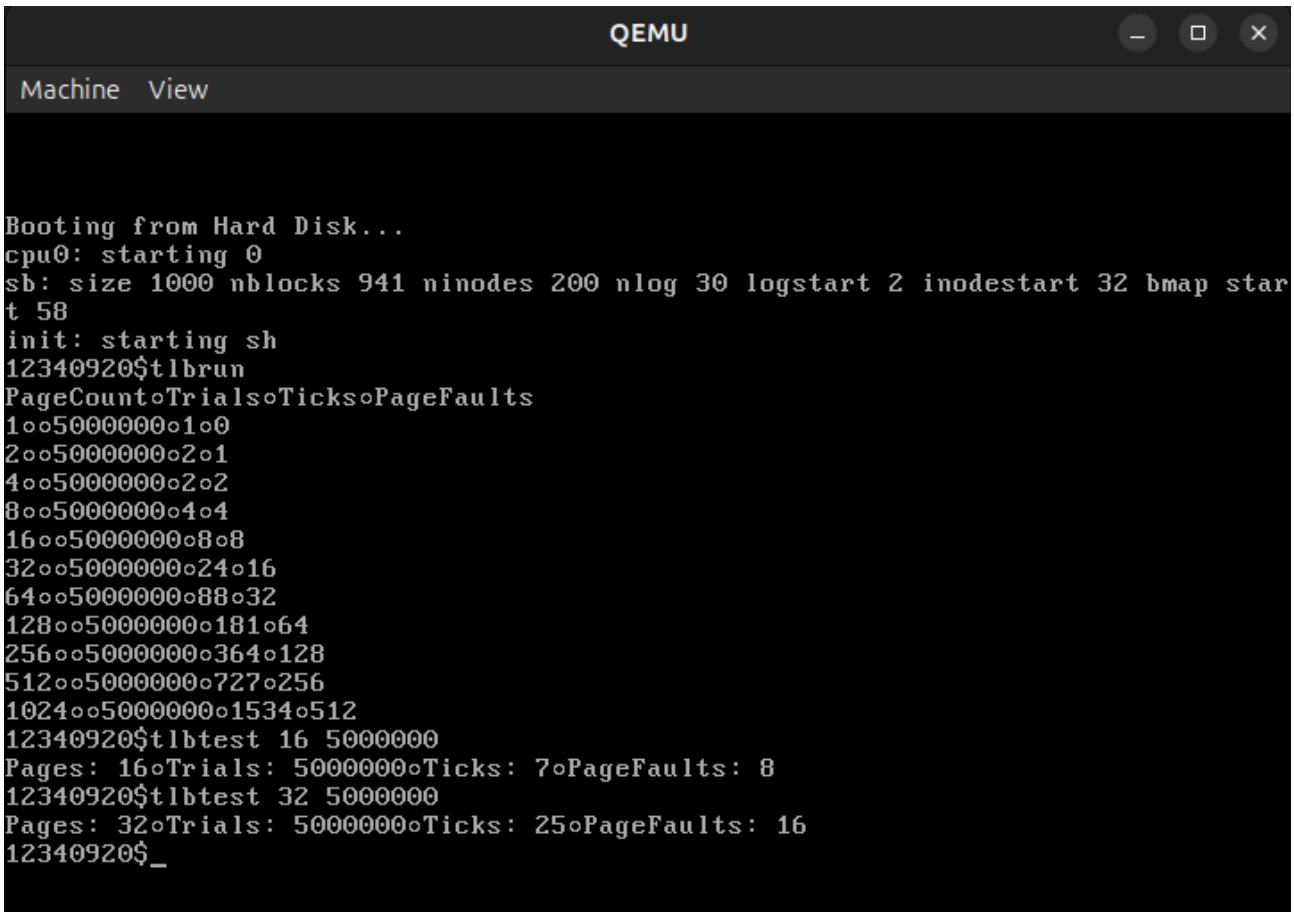
```
#include "types.h"
#include "user.h"

#define PAGESIZE 4096
#define MAXPAGES 1024

int main(int argc, char *argv[]) {
  if (argc < 3) {
```

```
    printf(1, "Usage: tlbtest <numpages> <trials>\n");
    exit();
  }
  int numpages = atoi(argv[1]);
  int trials = atoi(argv[2]);
  if (numpages < 1 || numpages > MAXPAGES) {
    printf(1, "numpages out of range (1..%d)\n", MAXPAGES);
    exit();
  }
  int jump = PAGESIZE / sizeof(int);
  int i, t;
  int *arr = (int *) sbrk(numpages * PAGESIZE);
  if (arr == (void*) -1) {
    printf(1, "sbrk failed for %d pages\n", numpages);
    exit();
  }
  int pf_before = getpagefaults();
  uint start_ticks = uptime();
  for (t = 0; t < trials; t++) {
    for (i = 0; i < (numpages / 2) * jump; i += jump) {
      arr[i] += 1;
    }
  }
  uint end_ticks = uptime();
  int pf_after = getpagefaults();
  printf(1, "Pages: %d\tTrials: %d\tTicks: %d\tPageFaults: %d\n",
         numpages, trials, end_ticks - start_ticks, pf_after - pf_before);
  exit();
}
```

13. Makefile

```
UPROGS=\
_tlbrun\
_tlbtest
```


harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Dow...
raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
12340920$tlbrun
PageCount Trials Ticks PageFaults
1 5000000 1 0
2 5000000 2 1
4 5000000 2 2
8 5000000 4 4
16 5000000 8 8
32 5000000 24 16
64 5000000 88 32
128 5000000 181 64
256 5000000 364 128
512 5000000 727 256
1024 5000000 1534 512
12340920$tlbtest 16 5000000
Pages: 16 Trials: 5000000 Ticks: 7 PageFaults: 8
12340920$tlbtest 32 5000000
Pages: 32 Trials: 5000000 Ticks: 25 PageFaults: 16
12340920$


QEMU
Machine View
Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
12340920$tlbrun
PageCount Trials Ticks PageFaults
1 5000000 1 0
2 5000000 2 1
4 5000000 2 2
8 5000000 4 4
16 5000000 8 8
32 5000000 24 16
64 5000000 88 32
128 5000000 181 64
256 5000000 364 128
512 5000000 727 256
1024 5000000 1534 512
12340920$tlbtest 16 5000000
Pages: 16 Trials: 5000000 Ticks: 7 PageFaults: 8
12340920$tlbtest 32 5000000
Pages: 32 Trials: 5000000 Ticks: 25 PageFaults: 16
12340920$_